

November 10th, 2025
Prof. Sieber
Geog 384

Assignment 3 - Mobile Tracking of VGI: *Report*

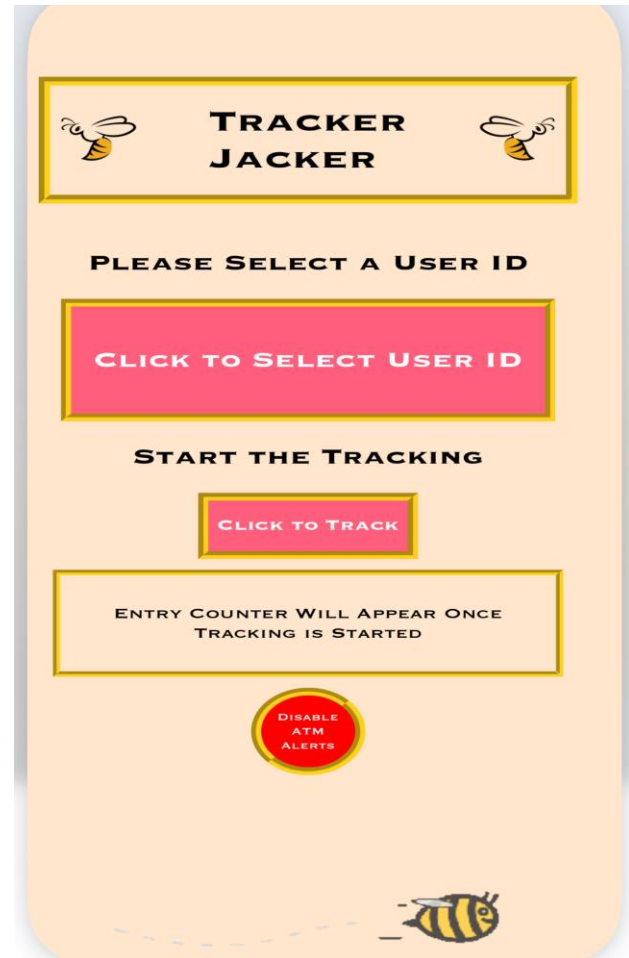
Group 3 – Kyra Meier, Ben Smith, Jack Ge

PASSWORD: neogeoweb

Part A – workflow

First, the user opens our group's general homepage at <https://neogeoweb.ca/group3/assignment3> which lists a quick directory to our mobile app tracking tool at [.../mobileapp](#) and our static path display with Mapbox at [.../display](#). We created the format in this way for consistency with our previous two assignments. Let's say the user wants to view the mobile app first. They click on the first URL and are taken to a login page where they must enter a password to access the mobile app, a feature we added as a security measure. They enter the case-sensitive password – neogeoweb – and then are redirected to another html page stored in this directory called [...authenticated.html](#), our main application. From here, they select their user ID – either Ben, Kyra, Jack or Instructor if they are the TA or Professor. This action sends the ID to the function that documents the selection by assigning it to variable UserId. If the user clicks outside of the options, the code reacts by removing the list of user IDs from view. We embedded some JavaScript to change the button styling to the user's name on a green background once it is selected.

Once you click on the button for your unique User ID, you must accept the browser pop up to permit the site to access your location 'while using app'. The next step is to start tracking. If a User ID has not been selected, the location tracking functions will not be triggered, and you will receive an alert to select a User ID first. If a user is selected, and you hit the "click to track" button, two alerts will be triggered – TrackUser() and StartTime(). The TrackUser() function uses the browser's navigator.geolocation.getCurrentPosition() method to get the current location



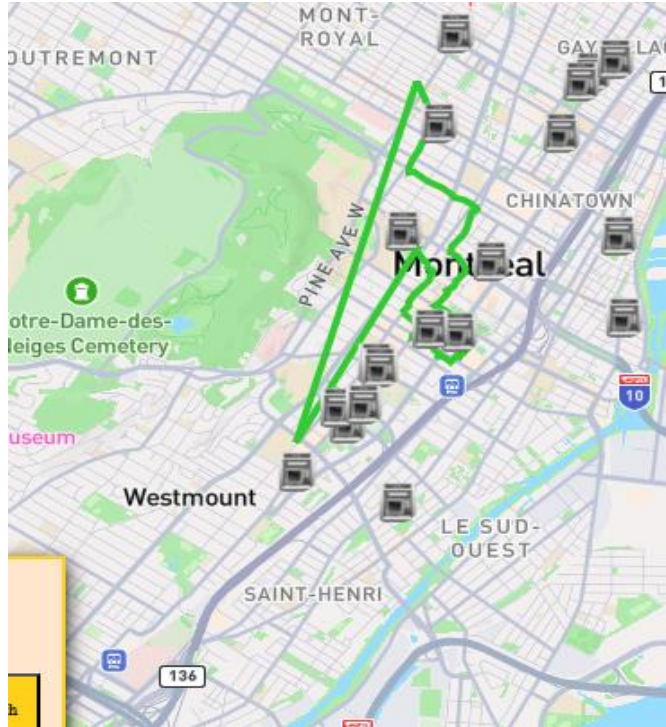
and a repeating timer using `setInterval()` automatically requests the user's location every 30 seconds (30000 milliseconds). The `StartTime()` function records the exact time that the user begins tracking and then displays that information on our webpage. Each time a user's x y location is obtained successfully, it calls the `success()` function.

The `success()` function makes up the bulk of our mobile app, only called on when the user is actively being tracked. It operates by getting the position of the user, assigning a new array to the selected user, and concatenating this array with the latitude, longitude, and the time of data collection via the `.push()` function. Below the success function, we call on the `writeToCV()` function so that this data is stored in our directory, not just temporarily kept on the console until the next page refresh. Once the data is written to the CSV, it will be converted to JSON via Ian's `.php` code.

Next, we have our ATM alert system that alerts the owner if they have approached within 50m of an ATM during their tracking session. This functionality works through calculating proximity of distance for the constant variable features of our ATM geojson. Once the coordinates of these features have been parsed in a for loop, the alert is triggered through an if statement that the distance between the local user coordinates and the ATM geometry feature is equal to that of our interest proximity – 50m. Once you have tracked yourself, your data has been stored, and you have received some alerts of proximity to an ATM, you have pretty much experienced our mobile app! The workflow is simple in structure, but becomes complex when looking at the number of nested functions. It is important to note the structure of our directory that is relatively unorganized compared to previous assignments, so that all the mobile app page can read and write seamlessly, something that “assets” folders and other file nesting would prohibit.

The display page is designed to read the tracking data collected from the main mobile app (<https://neogeoweb.ca/group3/assignment3/mobileapp/authenticated.html>) and visualize it on an interactive Mapbox map. When the page loads, it initializes a Mapbox map centered on downtown Montreal at coordinates -73.565, 45.502 with a zoom level of 12. In the event of the map's loading, the page sets up three separate GeoJSON data sources by loading JSON files named "Kyra.json", "Jack.json", and "Ben.json" - these files contain the location data that was previously collected and stored by the tracking page. For each user, the map creates a line layer with a unique color: hot pink for Kyra, royal blue for Jack, and lime green for Ben. All three layers are initially set to hidden with “visibility: none” so nothing appears on the map when it first loads. The line width for all paths is set to 4 pixels to make them easily visible on the map. A legend box positioned in the bottom-left corner of the map contains four buttons that let users control which tracking paths are displayed.

Each button calls a different function: showKyra(), showJack(), showBen(), or showAll(). When a user clicks on an individual person's button, the corresponding function sets that person's layer visibility to 'visible' while hiding the other two layers. For example, clicking "Kyra's path" makes only the pink line visible by setting Kyra's visibility to 'visible' and both Jack's and Ben's to 'none'. The "All paths" button is different because it sets all three layers to 'visible' at the same time, allowing users to see everyone's movement patterns overlaid on the same map. In addition, the user can see the ATM locations relative to each group's path because the html page reads in a separate script called "config.js" holding the geojson features for each ATM in the dataset.



The Tracker Jacker theme is taken from a particularly grueling Hunger Games feature, where unique half-robo, half-natural wasps take on to the scent of the Games players (Tributes) and tracks them and stings them, in some cases to death.

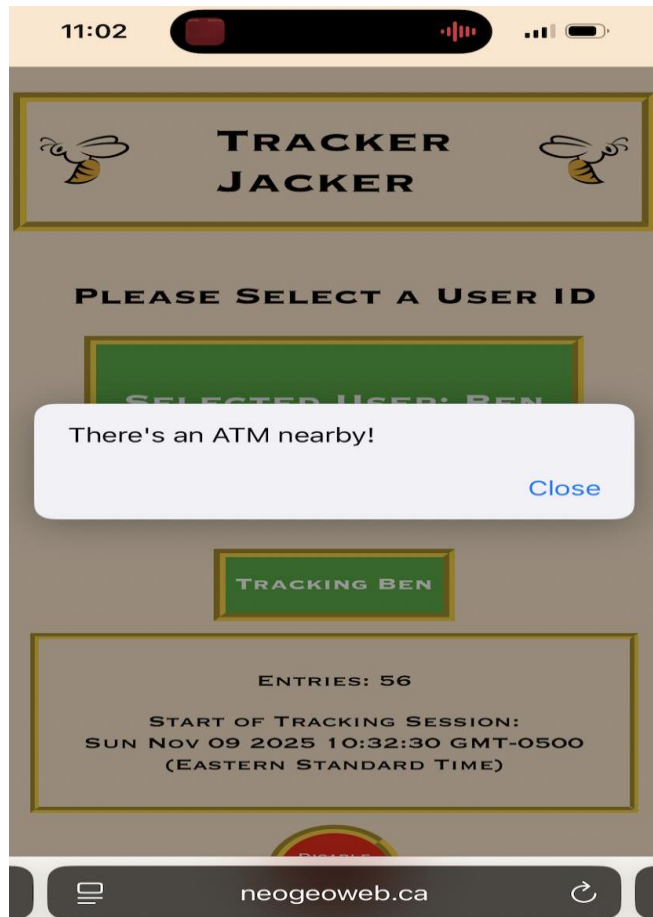
Part B – determining workflow

- a. Describe how your team determined the workflow (~1 page). “Well, we first thought about... We figured we needed to... We tried x and that didn’t work, so we tried y....”

The first aspect that we thought about was privacy. To ensure that only group 3 members (and the instructors) could log into our tracking page, we created a password protected html page. This login form prompted the user to input a password (“neogeoweb”) and sent an alert to deny access if incorrectly submitted. We tried also making the user input their username with the password but figured it would be easier to log the ID on the actual tracking html (to avoid asking to input the variable value twice). Once directed to the tracking page (upon a successful password entry), we needed to come up with a theme for the app. We chose to call our page “Tracker Jacker” after the genetically engineered wasps that pursue enemies from the dystopian novel Hunger Games. Much like these creatures, our website tracks the user (although not in the same harmful fashion). With the theme selected, we added images, borders, buttons, and colour schemes to match the Tracker Jacker inspiration. We then decided that we needed three main

buttons; one to select the user ID, a second to begin the tracking process, and a third to provide the user with the option to disable atm proximity alerts. We realized that obtaining the user ID was an important first step, as we needed to ensure that each of our respective tracings were recorded independently of each other. With the user ID selected (Ben, Kyra, Jack, or Instructor), the code records the specific person that it was preparing to track. Initially, we tried making the user type in their name or McGill ID, but we quickly realized that the case sensitivity would make it more difficult to assign the array. Plus, the dropdown menu looks more aesthetically pleasing. From there, we decided the next step would be to begin the tracking process for the individual. We also decided to include a visual below the tracking button displaying the exact time that the user began the tracking (in case they forget), and with the number of tracking entries that have been recorded (1 every 30 seconds). We tried to make a timer that began once the user started the tracking, but realized it would be simpler to just record the initial time since the user is bound to have an updating clock on the device that they are using to open the app. Additionally, the tracking methods on W3Schools only seemed to record the amount of time since January 1, 1970 in a timer format (which could have been converted if we wanted to make the code slightly more complex). Once the user is satisfied with their tracking entries, they may close the app. For the alert, we decided to keep it at every 30 seconds so as to not overcomplicate the code which for the tracking was already putting in entries every 30 seconds. We originally wanted to create a circle around every ATM location based on the data in the geojson but we quickly realized that it was going to be too complicated. We then decided to calculate the distance to every ATM location using the Haversine formula and if it found one within 50m of our current location, it would trigger the alert. We decided this was the best option because our geojson dataset wasn't very large meaning the brute force approach to finding nearby ATMs wasn't going to slow down the app too much.

Part C – alerts and location permissions



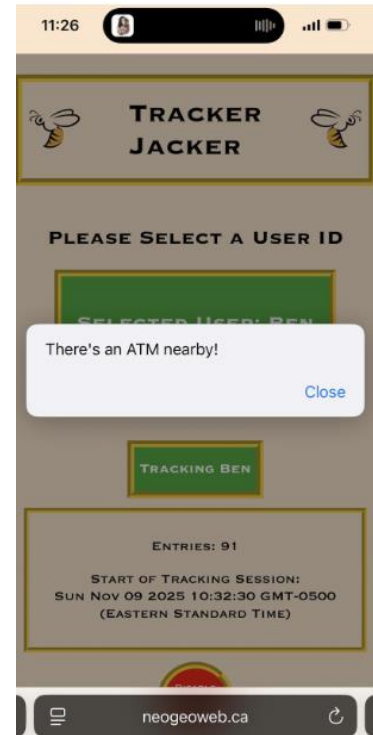
Using the location of the user, we decided to have the alert triggered if the user is within 50 meters of an ATM. To get the location of the user, we had to ensure that our web browser permissions were set to allow it to get access to the location. On the tracking page

(<https://neogeoweb.ca/group3/assignment3/mobileapp/authenticated.html>), the website was only going to start tracking when we clicked on the button to start tracking after choosing a user. Every 30 seconds, the website checks using the Haversine formula to calculate the distance between every ATM location and our current location. If it finds one that is within 50 meters of the user's location, it will stop the process of checking other ATMs and alert the user that there's an ATM nearby. The alert is made in a way where it only alerts the user if there's a ATM within 50 meters of the users location however, it doesn't tell the user which ATM is nearby meaning that even if there are multiple ATMs all within 50 meters of the user, it will only give the alert once. The alerts are set to go off about every 30 seconds if there is an ATM nearby which is also the same amount of time for each location entry to be recorded unless if the user chooses to turn the alerts off which in this case, the location entries will still continue being recorded, but the alerts won't happen. The ATMs locations were in a geojson format which contained information such as the location, name, opening hours, etc.

Despite being able to retrieve the user's location in most cases, there were some cases where the location wasn't able to be accurately determined. Such examples include being underground like in the metro where even though there is cellular signal, locations were unable to be determined from there which could lead to false alerts as the website uses our last known position. Another example of difficulty obtaining our location was when our cell signal was

weak where it was difficult to get a precise location of the user which could lead to inaccurate data being used by the website. Privacy settings on our device was the most common issue that we experienced as most of the time, our settings were set to not allow our browser to use our location meaning it was unable to fetch our location unless we changed the permissions.

Browser-based location tracking asks the user for permission every time it wants to use their location. Most of the time, the location is being tracked only when the website is open and the device is on. This has the benefit of providing the user with more privacy as they can choose when to stop the tracking. On the other hand, apps like Google Maps often continue tracking the user's location even when the device is off or if the user is in a different app. This is due to the apps continuously having access to the hardware of the device that provides the location meaning that it doesn't have to constantly ask the user for permission every set amount of time. The app can continue to access the user's location, even when the phone is off which is often used for functions such as navigation where constantly having the phone on is impractical for the user. Even though the user still has to give permission to the apps on their first use, there is no need to further permission approvals once the app has started location tracking unless the user goes and manually sets the settings to ask every time the app wants to use the location.



Part D – browser vs. Location permissions

Browser-based location tracking differs from the location permissions you grant in apps like Google Maps that get installed from the App Store in terms of the console versus the phone's operating system itself. Google Maps, for example, can store your data in a timeline because they have permission from the iPhone or Samsung OS to run “Always”, “When using”, or “Never”, depending on the set preferences. These types of location permissions are much more accurate because they can access the phone's GPS directly. In this way, they do not need to stop and ask permission from the browser, they can always track on a deep level. Meanwhile, the mobile app that we created and other websites require for the site to ask permission, which we learned resets when you close the browser or turn off your phone. It also only writes to the console, which means that they coordinates are only saved normally within the user session, and upon closing the session, the console log entries (your location information) disappears. The workaround for this is to write a file from the console entries, which is functional but far less robust than the genuine app store, local OS-based geolocation apps.

Regarding the use of a GPT, ChatGPT was used to help modify the login page code to stop the form from submitting so that we could instead compare the password with a reference password. It was also used to generate simple CSS styling statements and to help us understand the `setInterval()` function to capture the user's location every 30 seconds. We also used the GPT for display page styling, asking for CSS code for a proportionate legend in the theme of our app.

Personal Reflections

Kyra

For me, the most challenging part of the assignment was organizing the many files associated with our project on Cyberduck. Of this role and of exporting the ATM dataset, the creation of the display map and website with path visualization, and the csv/JSON writing in Step 4, I found the Cyberduck most attention-demanding to ensure I wouldn't overwrite something. It was frankly shocking to me how easy and powerful it is for the site consistently and indefinitely obtain a user's identified, timestamped and precise location. When the popup asks you, "can this site use your location?" before this class thought that the site gets some general implication of my 'location', but I was not educated to know that the site can track your exact spatial coordinates down to the most detailed degree at what could be every single second, and send that data to an external database which could be retained permanently. That the site has no obligation to tell you this, and it only took us a couple of lines of code to write the location data into a cumulative geospatial file for each user. In our case, the web app very clearly notifies the user that the point of them sharing their location means that they're going to be tracked, but most sites don't say that, just some vague thing about "getting better service". This aligns with what we discussed about VGI, where almost all monetized content used in web dev is volunteered, but not often without the "volunteer's" full understanding or informed consent (Sieber, Lecture, 2025). Along these lines, our app is absolutely a form of surveillance, not crowdsourcing. The only reason this "tracker" is not outright spyware is because the webpage tells the user very clearly that they are being surveilled and how. The tracker does not collect your location data if you turn off your phone screen or if you close the browser or close the mobile app page. However, if you leave the tab open and keep your phone screen, passively leave it running, the app will continue tracking. We didn't write a time-out function into the code so potentially it could track forever. The way I was operating the app during testing was by turning off the location permissions when I wanted it to stop tracking me. I think that the operability is very problematic and speaks to the incredible lack of regulation on sites to collect and store your location data.

Ben

Personally, the most challenging part of the assignment was figuring out a clear system for how to track the specific user and store their subsequent latitude, longitude, and time data of

each 30 second record. This process involved creating an array of objects for each potential user (Ben, Kyra, Jack) and then populating the respective array with an object {lat,lon,time} every 30 seconds. While this sounds not extremely complicated to appear, formatting this idea and implanting it into the code took many hours and trials to get a successful output. Furthermore, I unfortunately did not realize that running the html locally while testing the code would result in asking the user every 30 seconds to re-confirm their consent to share their location. As a result, I spent another number of hours troubleshooting this issue, playing around with the loop structure and alternative tracking functions. Luckily, Ian reminded me that this issue would be resolved if I uploaded the html to the neogeoweb cyberduck (problem solved!).

I was very surprised at how easy it was to collect a user's space-time signature. The fact that most of the code information can be found on publicly accessible and free websites like W3Schools was alarming for me. The lack of knowledge required to construct a tracking page is very surprising and is slightly worrying. Although our assignment is more practical and innocent, in the wrong hands this technology could easily be incorporated to track users without their knowledge or consent. A publication by Papadogiannakis et al. (2021) found that websites use "modern forms of tracking even before users had the opportunity to register their choice with respect to cookies". The fact that the resources to track users are more accessible than I previously thought, coupled with the advancement in tracking without consent from companies, is particularly startling. When we first discussed the concept of data brokers in lecture, I was under the impression that they were highly trained professionals. However, it seems like location tracking can be learnt without an extensive educational background.

The app developed in this assignment is an example of crowdsourcing. While I am tempted to say surveillance, the user must click on a series of buttons to get their location tracked. Since the user must select their unique user profile and initiate the tracking themselves, they are actively volunteering for their spatial data. It could be argued that the user may not be certain about what is done with their data, and that surveilling could take place in the background of the application or by sending information to third parties. However, this app would not track the user without their consent and knowledge. If our page monitored each user while the app was closed and without their active participation, this would most certainly be a form of surveillance.

The tracker does not keep working if the phone is turned off, if the browser is closed, or if the browser tab is not open. However, if the user forgets to turn off their device with the page still open, the app will continue to passively collect data (without a maximum runtime included in the code). This may be problematic if the user forgets to manually close the browser tab after intentionally tracking themselves, which would lead to involuntary data collection. This would be beneficial if a company owned our app and was able to receive more data from the user.

Jack

For me, the most challenging part was figuring out how I was going to be able to calculate the distance between the user and the ATMs locations. During the process of implementing a system to trigger the alerts, I had to constantly be careful of the units being used as we wanted to

measure the distances in meters whereas the location data was in degrees. I am both surprised and not surprised that it's easy to collect a user's space-time signature because I knew that it was possible to collect a user's location however, I didn't realize that many times, only a few lines of code was needed to collect the user's location. In a case involving a user's expectations of privacy, it was noted by the judge that the mass surveillance of electronic devices raises some concern of privacy invasion. In another one, there was mention of users accusing companies of continuing to track their locations despite the user supposedly denying the company from collecting more information about the user's location (Claypoole & Balough, 2012). These issues highlight some of the problems we face in today's digital world as it has become very easy for companies to collect information about us while not being fully transparent on what's happening behind the scenes. For our app, I consider it to be surveillance because it constantly notes down the user's location every 30 seconds until the website is refreshed or closed. This also highlights the issue with transparency as based on what our app is doing, it is continuously tracking the user's location behind the scenes, even if some people might think that it has stopped doing so. Even though we decided to add information about the number of entries made since the start of tracking, this information can easily be hidden by companies leading to unclear information being provided to the user. This can be especially problematic when it comes to privacy issues as Claypoole & Balough (2012) noted, many individual lawsuits were filed against companies for supposedly violating personal privacy rights.

Personally, I'm not surprised that the tracker stops working when our phone screen is turned off or if we close the browser because we have it programmed so that it's only going to work if the user is on the webpage live throughout the process. Even if we choose to go to another app, the website will pause it's tracking until we're back onto the website. The benefits of this approach is that the user can know when their location is being tracked and can easily stop the tracking when they feel like it. On the other hand, for a project like this, it becomes inconvenient in that we have to constantly "wake up" our phone to make sure it keeps tracking our location and recording it.

Reference:

- Claypoole, T. F., & Balough, R. C. (2012). Developments in the Law Concerning Geolocational Privacy. *The Business Lawyer*, 68(1), 197–204. <http://www.jstor.org/stable/23527084>
- Papadogiannakis, E., Papadopoulos, P., Kourtellis, N., & Markatos, E. P. (2021). User Tracking in the Post-cookie Era: How Websites Bypass GDPR Consent to Track Users. *Proceedings of the Web Conference 2021*, 2130–2141. <https://doi.org/10.1145/3442381.3450056>